

RISC-V Reference

- Register Names

Register	ABI Name	Description	Saver
x0	zero	Constant value 0	-
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	-
x4	tp	Thread pointer	-
x5-7	t0-2	Temporary registers	Caller
x8	s0 / fp	Saved register / Frame pointer	Callee
x9	s1	Saved register	Callee
x10-11	a0-1	Function arguments / Return values	Caller
x12-17	a2-7	Function arguments	Caller
x18-27	s2-11	Saved registers	Callee
x28-31	t3-6	Temporary registers	Caller

imm: signed immediate in `imm11:0`
uimm: 5-bit unsigned immediate in `imm4:0`
upimm: upper bits of immediate, in `imm31:12`
Address: memory address:
`rs1 + SignExt(imm11:0)`
[Address]: data at memory location Address
BTA: branch target address:
`PC + SignExt({imm12:1, 1'b0})`
JTA: jump target address:
`PC + SignExt({imm20:1, 1'b0})`
label: text indicating instruction address
SignExt: value sign-extended to 32 bits
ZeroExt: value zero-extended to 32 bits
(U): instruction operates on unsigned data
csr: control and status register

- Instruction Formats

31	25	24	20	19	15	12	11	7	6	0	
funct7	rs2	rs1	funct3	rd	opcode						R-Type
imm _{11:0}		rs1	funct3	rd	opcode						I-Type
imm _{11:5}	rs2	rs1	funct3	imm _{4:0}	opcode						S-Type
imm _{12,10:5}	rs2	rs1	funct3	imm _{4:1,11}	opcode						B-Type
				rd	opcode						U-Type
				rd	opcode						J-Type

- RV32I: Integer Instructions

opcode	funct3	funct7	Type	Instruction	Description	Operation
0000011	000	-	I	lb rd, imm(rs1)	load byte	rd = SignExt([Address] _{7:0})
0000011	001	-	I	lh rd, imm(rs1)	load half	rd = SignExt([Address] _{15:0})
0000011	010	-	I	lw rd, imm(rs1)	load word	rd = [Address] _{31:0}
0000011	100	-	I	lbu rd, imm(rs1)	load byte (U)	rd = ZeroExt([Address] _{7:0})
0000011	101	-	I	lhu rd, imm(rs1)	load half (U)	rd = ZeroExt([Address] _{15:0})
0010011	000	-	I	addi rd, rs1, imm	add imm.	rd = rs1 + SignExt(imm)
0010011	001	0000000*	I	slli rd, rs1, imm	shift left logical imm.	rd = rs1 << uimm
0010011	010	-	I	slti rd, rs1, imm	set less than imm.	rd = (rs1 < SignExt(imm)) ? 1:0
0010011	011	-	I	sltiu rd, rs1, imm	set less than imm. (U)	rd = (rs1 < SignExt(imm)) ? 1:0
0010011	100	-	I	xori rd, rs1, imm	xor imm.	rd = rs1 ^ SignExt(imm)
0010011	101	0000000*	I	srlr rd, rs1, imm	shift right logical imm.	rd = rs1 >> uimm
0010011	101	0100000*	I	srai rd, rs1, imm	shift right arithmetic imm.	rd = rs1 >>> uimm
0010011	110	-	I	ori rd, rs1, imm	or imm.	rd = rs1 SignExt(imm)
0010011	111	-	I	andi rd, rs1, imm	and imm.	rd = rs1 & SignExt(imm)
0010111	-	-	U	auipc rd, upimm	add upper imm. to PC	rd = {upimm, 12'b0} + PC
0100011	000	-	S	sb rs2, imm(rs1)	store byte	[Address] _{7:0} = rs2 _{7:0}
0100011	001	-	S	sh rs2, imm(rs1)	store half	[Address] _{15:0} = rs2 _{15:0}
0100011	010	-	S	sw rs2, imm(rs1)	store word	[Address] _{31:0} = rs2
0110011	000	0000000	R	add rd, rs1, rs2	add	rd = rs1 + rs2
0110011	000	0100000	R	sub rd, rs1, rs2	sub	rd = rs1 - rs2
0110011	001	0000000	R	sll rd, rs1, rs2	shift left logical	rd = rs1 << rs2 _{4:0}
0110011	010	0000000	R	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2) ? 1:0
0110011	011	0000000	R	sltu rd, rs1, rs2	set less than imm.	rd = (rs1 < rs2) ? 1:0
0110011	100	0000000	R	xor rd, rs1, rs2	xor	rd = rs1 ^ rs2
0110011	101	0000000	R	srl rd, rs1, rs2	shift right logical	rd = rs1 >> rs2 _{4:0}
0110011	101	0100000	R	sra rd, rs1, rs2	shift right arithmetic	rd = rs1 >>> rs2 _{4:0}
0110011	110	0000000	R	or rd, rs1, rs2	or	rd = rs1 rs2
0110011	111	0000000	R	and rd, rs1, rs2	and	rd = rs1 & rs2
0110111	-	-	U	lui rd, upimm	load upper imm.	rd = {upimm, 12'b0}
1100011	000	-	B	beq rs1, rs2, label	branch if =	if (rs1 == rs2) PC = BTA
1100011	001	-	B	bne rs1, rs2, label	branch if !=	if (rs1 != rs2) PC = BTA
1100011	100	-	B	blt rs1, rs2, label	branch if <	if (rs1 < rs2) PC = BTA
1100011	101	-	B	bge rs1, rs2, label	branch if ≥	if (rs1 ≥ rs2) PC = BTA
1100011	110	-	B	bltu rs1, rs2, label	branch if < (U)	if (rs1 < rs2) PC = BTA
1100011	111	-	B	bgeu rs1, rs2, label	branch if ≥ (U)	if (rs1 ≥ rs2) PC = BTA
1100111	000	-	I	jalr rd, rs1, imm	jump and link register	PC = rs1 + SignExt(imm) rd = PC + 4
1101111	-	-	J	jal rd, label	jump and link	PC = JTA, rd = PC + 4

*Encoded in the upper seven bits of the immediate field (instr_{31:25})

- Multiply and Divide Instructions (M Extension)

opcode	funct3	funct7	Type	Instruction	Description	Operation
0110011	000	0000001	R	mul rd, rs1, rs2	multiply	rd = (rs1 * rs2) _{31:0}
0110011	001	0000001	R	mulh rd, rs1, rs2	multiply high (S) (S)	rd = (rs1 * rs2) _{63:32}
0110011	010	0000001	R	mulhsu rd, rs1, rs2	multiply high (S) (U)	rd = (rs1 * rs2) _{63:32}
0110011	011	0000001	R	mulhu rd, rs1, rs2	multiply high (U) (U)	rd = (rs1 * rs2) _{63:32}
0110011	100	0000001	R	div rd, rs1, rs2	divide (S)	rd = rs1 / rs2
0110011	101	0000001	R	divu rd, rs1, rs2	divide (U)	rd = rs1 / rs2
0110011	110	0000001	R	rem rd, rs1, rs2	remainder (S)	rd = rs1 % rs2
0110011	111	0000001	R	remu rd, rs1, rs2	remainder (U)	rd = rs1 % rs2

- Privileged / CSR Instructions

opcode	funct3	Type	Instruction	Description	Operation
1110011	000	I	ecall	transfer control to OS (imm=0)	
1110011	000	I	ebreak	transfer control to debugger (imm=1)	
1110011	000	I	uret	return from user exception (rs1=0,rd=0,imm=2)	PC = uepc
1110011	000	I	sret	return from supervisor exception (rs1=0,rd=0,imm=258)	PC = sepc
1110011	000	I	mret	return from machine exception (rs1=0,rd=0,imm=770)	PC = mepc
1110011	001	I	csrrw rd, csr, rs1	CSR read/write (imm=CSR number)	rd = csr csr = rs1
1110011	010	I	csrrs rd, csr, rs1	CSR read/set (imm=CSR number)	rd = csr csr = csr rs1
1110011	011	I	csrrc rd, csr, rs1	CSR read/clear (imm=CSR number)	rd = csr csr = csr & rs1
1110011	101	I	csrrwi rd, csr, uimm	CSR read/write imm. (imm=CSR number)	rd = csr csr = ZeroExt(uimm)
1110011	110	I	csrrsi rd, csr, uimm	CSR read/set imm. (imm=CSR number)	rd = csr csr = csr ZeroExt(uimm)
1110011	111	I	csrrci rd, csr, uimm	CSR read/clear imm. (imm=CSR number)	rd = csr csr = csr & ~ZeroExt(uimm)

For privileged / CSR instructions, the 5-bit unsigned immediate, uimm, is encoded in the rs1 field.

- Pseudo Instruktionen

Pseudo Instruction	Base Instruction(s)	Description	Operation
nop	addi x0, x0, 0	no operation	
li rd, imm _{11:0}	addi rd, x0, imm _{11:0}	load 12-bit immediate	rd = SignExtend(imm _{11:0})
li rd, imm _{31:0}	lui rd, imm _{31:12} * addi rd, rd, imm _{11:0}	load 32-bit immediate	rd = imm _{31:0}
mv rd, rs1	addi rd, rs1, 0	move	rd = rs1
not rd, rs1	xori rd, rs1, -1	one's complement	rd = ~rs1
neg rd, rs1	sub rd, x0, rs1	two's complement	rd = -rs1
seqz rd, rs1	sltiu rd, rs1, 1	set if = 0	rd = (rs1 == 0)
snez rd, rs1	sltu rd, x0, rs1	set if != 0	rd = (rs1 != 0)
sltz rd, rs1	slt rd, rs1, x0	set if < 0	rd = (rs1 < 0)
sgtz rd, rs1	slt rd, x0, rs1	set if > 0	rd = (rs1 > 0)
beqz rs1, label	beq rs1, x0, label	branch if = 0	if (rs1 == 0) PC = label
bnez rs1, label	bne rs1, x0, label	branch if != 0	if (rs1 != 0) PC = label
blez rs1, label	bge x0, rs1, label	branch if ≤ 0	if (rs1 ≤ 0) PC = label
bgez rs1, label	bge rs1, x0, label	branch if ≥ 0	if (rs1 ≥ 0) PC = label
bltz rs1, label	blt rs1, x0, label	branch if < 0	if (rs1 < 0) PC = label
bgtz rs1, label	blt x0, rs1, label	branch if > 0	if (rs1 > 0) PC = label
ble rs1, rs2, label	bge rs2, rs1, label	branch if ≤	if (rs1 ≤ rs2) PC = label
bgt rs1, rs2, label	blt rs2, rs1, label	branch if >	if (rs1 > rs2) PC = label
bleu rs1, rs2, label	bgeu rs2, rs1, label	branch if ≤ (U)	if (rs1 ≤ rs2) PC = label
bgtu rs1, rs2, label	btlu rs2, rs1, label	branch if > (U)	if (rs1 > rs2) PC = label
j label	jal x0, label	jump	PC = label
jal label	jal ra, label	jump and link	PC = label, ra = PC + 4
jr rs1	jalr x0, rs1, 0	jump register	PC = rs1
jalr rs1	jalr ra, rs1, 0	jump and link register	PC = rs1, ra = PC + 4
ret	jalr x0, ra, 0	return from function	PC = ra
call label	jal ra, label	call nearby function	PC = label, ra = PC + 4
call label	auipc ra, offset _{31:12} * jalr ra, ra, offset _{11:0}	call far away function	PC = PC + offset ra = PC + 4
la rd, symbol	auipc rd, symbol _{31:12} * addi rd, rd, symbol _{11:0}	load address of global variable	rd = PC + symbol
l{b h w} rd, symbol	auipc rd, symbol _{31:12} * l{b h w} rd, symbol _{11:0} (rd)	load global variable	rd = [PC + symbol]
s{b h w} rs2, symbol, rs1	auipc rs1, symbol _{31:12} * s{b h w} rs2, symbol _{11:0} (rs1)	store global variable	[PC + symbol] = rs2
csrr rd, csr	csrrs rd, csr, x0	read CSR	rd = csr
csrw csr, rs1	csrrw x0, csr, rs1	write CSR	csr = rs1

if imm₁₁ == 1, the upper immediate_{31:12} must be incremented by 1.